



Exploiting Artificial Intelligence in Software Development

Author **Bc. Kristián Rusnák**

Supervisor **Ing. Lukáš Radoský**

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Master's Thesis Progress Seminar · Summer 2026

Motivation & Problem Statement



1



Massive Log Volume

Modern systems generate billions of log lines daily

2



Manual Inspection

Infeasible at scale — humans can't keep up

3



Anomaly Detection

Critical for reliability, but existing tools lack interpretability

4



The Goal

Detect anomalies AND explain them in human-readable form

Thesis Goal



Build a general-purpose, user-friendly agentic AI framework

for log anomaly detection — with classification AND natural language explanation



Generalizability

KEY PRINCIPLE

Works across different log formats and system types without retraining



Minimal Fine-Tuning

KEY PRINCIPLE

No (or near-zero) fine-tuning required — prompt-driven by design



Robustness

KEY PRINCIPLE

Resilient to log format changes and distribution shifts

Literature Review

5 KEY PAPERS SURVEYED

		DETECTION SCOPE	APPROACH TYPES
1	LLM-LADE Classification + annotated explanations	Explanation and Detection	LLM-based Framework
2	LogLLM LLM-based log parsing & detection	Explanation and Detection	LLM-based Framework
3	LogRESP-Agent Agent-based log reasoning pipeline	Detection	Agentic
4	Prompt Pattern Catalog Prompt engineering patterns for ChatGPT	—	Prompting
5	LogGPT GPT-based anomaly detection in system logs	Detection	LLM-based Framework

KEY FINDING



Most works focus on classification only

Explanation of anomaly root causes remains largely unexplored — this is the gap we try to address.

APPROACH TYPES

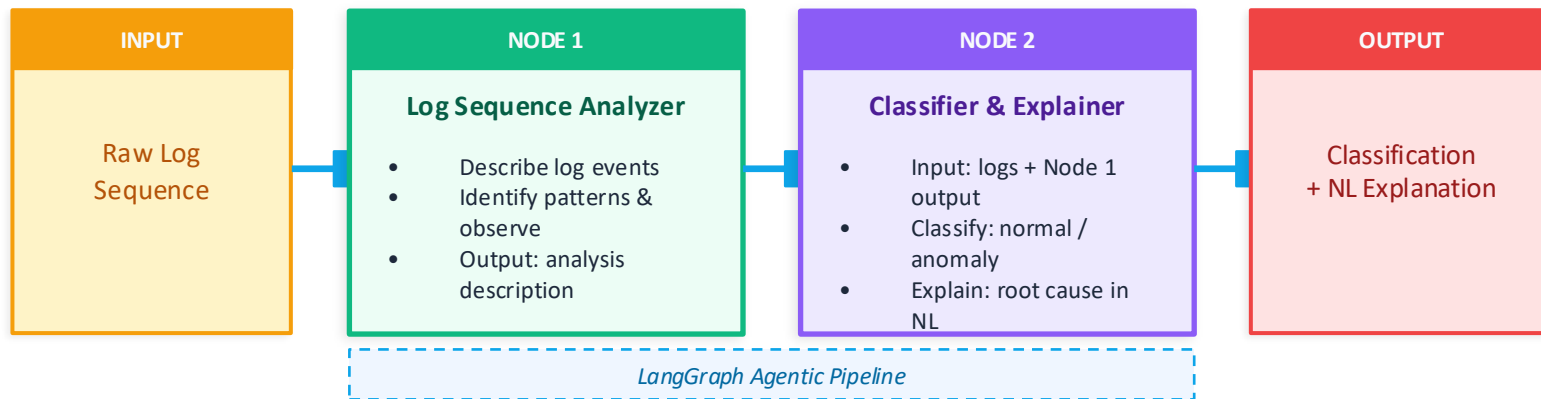
- LLM-based Framework
- Agentic
- Prompting

DETECTION SCOPE

- Detection
- Explanation and Detection

System Architecture

First Prototype — LangGraph Agentic Pipeline



Python

LangChain

LangGraph

Model-Agnostic

MCP / Tools (planned)

Datasets

LogHub Benchmark · 3 datasets · 30 annotated sequences each (LLM-LADE)

HDFS

Hadoop Distributed File System

SOURCE

Distributed storage nodes

SIZE

11 M lines

ANOMALY RATE

~2.9%

EXAMPLE LOGS

```
Receiving block <*> src: <*> dest: <*>
```

```
BLOCK* NameSystem.allocateBlock: <*> <*>
```

BGL

Blue Gene/L Supercomputer

SOURCE

BlueGene/L system logs

SIZE

4.7 M lines

ANOMALY RATE

~7.4%

EXAMPLE LOGS

```
instruction cache parity error corrected
```

```
ddr errors detected and corrected
```

Thunderbird

Sandia National Labs Cluster

SOURCE

Large-scale HPC cluster

SIZE

211 M lines

ANOMALY RATE

~0.7%

EXAMPLE LOGS

```
error detected on node <*>
```

```
kernel: <*>: eth0: transmit timed out
```

Benchmarking (Best)

First Prototype Evaluation · Thunderbird Dataset

① Classification Performance

94.44%

F1 Score

100%

Recall

89.47%

Precision

93.33%

Accuracy

② Explanation Quality (text-embedding-3-large)

0.8579

Best Similarity

Top performing sequence

0.7774

Average Similarity

Embedder cosine score

0.6971

Worst Similarity

Lowest performing sequence

Benchmarking (Worst)

First Prototype Evaluation · HDFS Dataset

① Classification Performance

63.64%

F1 Score

50%

Recall

87.50%

Precision

73.33%

Accuracy

② Explanation Quality (text-embedding-3-large)

0.8364

Best Similarity

Top performing sequence

0.7293

Average Similarity

Embedder cosine score

0.6427

Worst Similarity

Lowest performing sequence

Data Augmentation

08

Problem: only 30 annotated sequences per dataset — insufficient for robust evaluation

Approach 1 · In progress

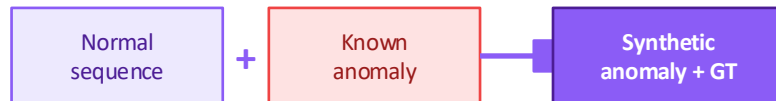
LLM Few-Shot Synthetic Generation



- Script implemented
- First synthetic data already generated
- Uses 3–5 real examples as prompt context
- Generates varied log sequences with labels & explanations
- Need validation strategy to verify generated datasets (log sequence correctness, classification accuracy, and cause explanation quality)

Approach 2 · Planned

Anomaly Injection into Normal Sequences



- Inject known abnormal messages into normal logs
- Creates synthetic anomalies with verifiable root causes
- Ground truth labels are deterministic — no annotation needed
- Enables scalable, controllable evaluation

Next Steps

Planned work for thesis completion

09



01

Validate & Expand Dataset

Review augmented data quality · scale to all 3 datasets

02

Tool Use Support

Extend prototype with MCP / tool calling capabilities

03

Integrate LogLLM

Add full LLM-based log detection framework as an agentic tool

04

Design Pipeline Tools

Build dedicated tools for log parsing, pattern search & retrieval

05

Re-evaluate System

Run full benchmark on improved pipeline with augmented data

Summary

Exploiting Artificial Intelligence in Software Development

COMPLETED

Literature Review

5 key papers surveyed

System Architecture

LangGraph 2-node pipeline

Benchmarking

Thunderbird and HDF5

Data Augmentation

Few-shot synthesis in progress

PLANNED

Expand dataset

Tool use / MCP

Integrate LogLLM

Re-evaluate

Expected contribution: a general-purpose, model-agnostic agentic framework for log anomaly detection that not only classifies but explains root causes in natural language.

Thank You

Questions?

AUTHOR	Bc. Kristián Rusnák
SUPERVISOR	Ing. Lukáš Radoský
FACULTY	FMFI · Comenius University
SEMESTER	Summer 2026